ENGINEX — Unified Engineering Simulation & Analysis Platform

Technical Report

1. Project Overview

ENGINEX Workbench is a desktop-oriented engineering analysis application combining geometry inspection, thermal finite-element analysis, performance benchmarking, embedded-device interface simulation and a backpropagation demonstration. It uses a Next.js static frontend and a C++17 backend exposed through a local HTTP API. The backend owns parsing, validation, numerical computation and job execution.

The Windows launcher starts the backend and opens the workbench. Long-running thermal and benchmark operations use background jobs with status reporting and cancellation. Hardware devices shown in the application are simulated; no physical MCU, FPGA or debug adapter connection is claimed.

2. System Architecture

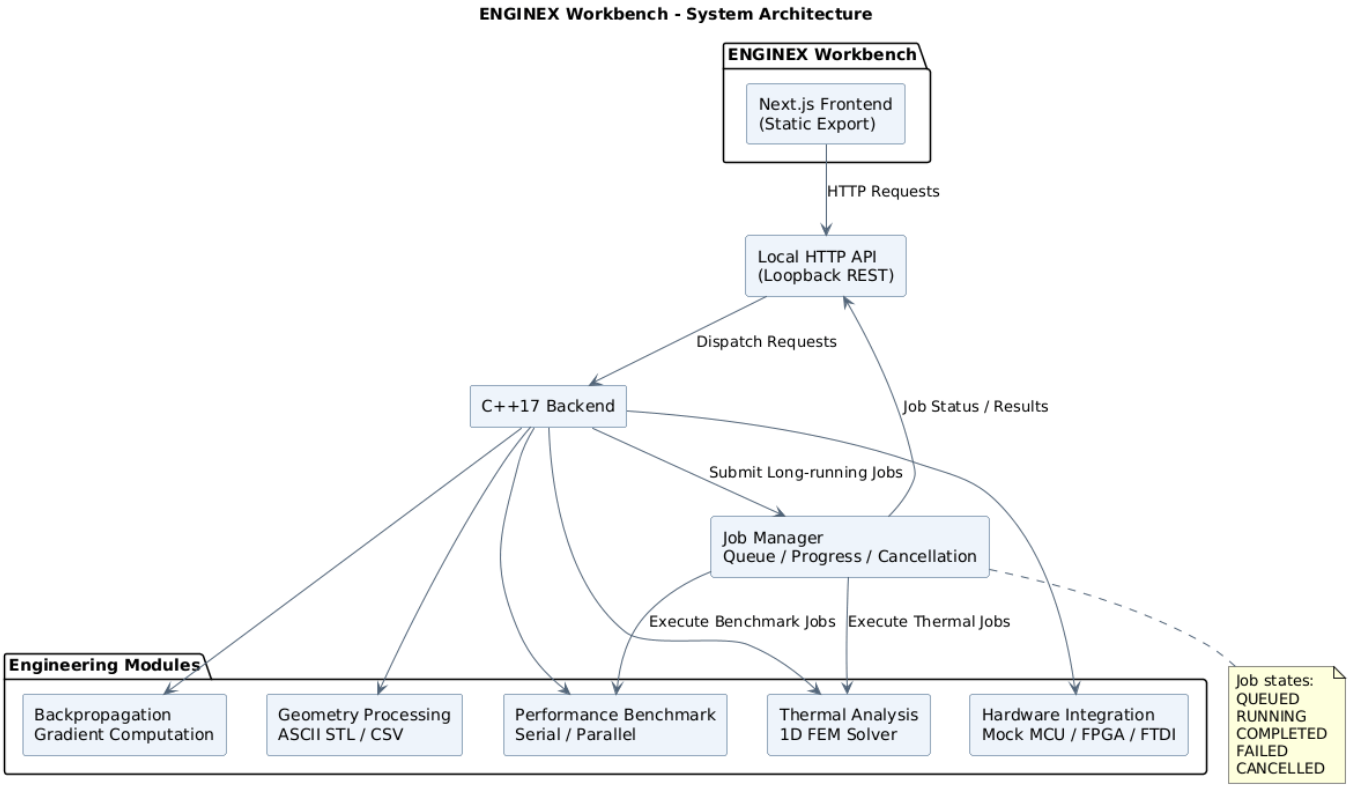


Figure: ENGINEX Workbench system architecture.

3. Geometry and Data Handling

The Geometry view supports inspection of ASCII STL geometry and CSV nodal temperature data. Parsing and validation are performed by the C++ backend; the frontend presents the geometry or point-cloud information. The supplied STL fixture demonstrates triangle parsing and geometry display.

The CSV point-cloud view maps node coordinates and temperature values. The example temperature file is described as a synthetic fixture in the project materials; it must not be represented as a verified commercial-solver export without provenance evidence.

3.1 Input Validation

- ASCII STL is parsed as text and checked for the expected facet and vertex structure.
- CSV input is checked for required columns and usable numeric values.
- Malformed or unsupported input should produce a visible error.
- Do not claim binary STL or arbitrary solver-result format support unless implemented.

3.2 Evidence

The separate Screenshot Walkthrough PDF contains the ASCII STL and nodal temperature point-cloud views. Screenshots demonstrate representative states; they do not replace parser tests or prove sample-data provenance.

4. Thermal Analysis: 1D Finite-Element Model

The thermal module solves steady one-dimensional conduction in a uniform rod using two-node linear elements. Parameters: $L=1$ m, $A=1\times10^{-4}$ m², $k=10$ W/(m·K), $q=10{,}000$ W/m³, $T(0)=100$°C and $T(L)=20$°C. The governing equation is $kA\,d^2T/dx^2 + qA = 0$.

The analytical solution is $T(x)=100+420x-500x^2$ °C. The FE system assembles element conductivity matrices and source vectors, applies Dirichlet boundary conditions and solves for nodal temperatures.

4.1 Mesh Convergence

For this constant-property case, FE nodal values agree with the analytical solution at the nodes. Piecewise-linear interpolation differs between nodes. Maximum midpoint interpolation error is $qh^2/(8k)$, where $h=L/N$.

Elements	h (m)	Max midpoint error (°C)
10	0.1	1.25
20	0.05	0.3125
40	0.025	0.078125
80	0.0125	0.01953125

Error decreases by a factor of four whenever element count doubles, consistent with the h^2 interpolation-error relationship.

5. Performance and Multicore Benchmarking

The performance module compares serial execution with CPU-parallel batch execution. A fixed seed makes the generated workload reproducible. A bounded worker pool is capped by logical CPU availability. The benchmark design uses warm-up and five measured repetitions, with medians used for comparison.

5.1 Measurement Method

- Compute-only timing is separated from application launch-to-ready time.
- The same generated workload is used for serial and parallel comparisons.
- Speedup = serial median time / parallel median time.
- Parallel efficiency = speedup / worker count.
- Latest regression reported serial/parallel maximum temperature difference no greater than 1×10^{-8}°C.

5.2 Interpretation

Parallel execution is not automatically faster. Small workloads can be dominated by scheduling, synchronization and thread-management overhead. An observed small workload ran slower in parallel. A larger workload showed approximate speedups of 1.46×, 1.68× and 1.73× for two, three and four workers. These are observed results, not universal guarantees.

Timing depends on workload size, seed, compiler and machine. Exact timing tables should be copied from the final exported CSV rather than reconstructed from rounded interface observations.

6. Embedded Hardware Integration Design6.1 Device Roles and Interfaces

The Hardware view demonstrates a modular integration architecture for three embedded-system components: a microcontroller (MCU), a field-programmable gate array (FPGA), and a USB debug adapter. The objective is to provide a unified software interface for device discovery, connection management, telemetry monitoring and fault handling.

The current ENGINEX implementation uses mock adapters and simulated telemetry. Device cards, connection states and fault injection demonstrate the software interface and interaction workflow. No physical hardware communication is claimed.

6.1 Device Roles and Interfaces

Component	Proposed role	Interface concept
STM32F401RE Nucleo	MCU target	UART telemetry; SWD target debug
Lattice iCE40UP5K	FPGA target	Digital I/O/programming path
FTDI FT2232H	USB debug adapter	USB host; configurable JTAG/SPI/UART

Conventional JTAG uses TMS, TCK, TDI and TDO. ARM SWD uses SWDIO and SWCLK; these are not interchangeable with the full JTAG signal set. Verify board-specific pinouts and voltage levels against manufacturer schematics before wiring.

6.2 Fault Injection and Recovery

The current implementation includes simulated fault injection to demonstrate device communication error handling. For example, a simulated MCU timeout generates an error notification while the application updates the device status and displays the recovery state.

The mock adapter can subsequently resume telemetry updates, demonstrating the software-level recovery workflow. This validates application behaviour only; it does not establish physical fault protection or recovery on real hardware.

7. Backpropagation Demonstration

The Backpropagation view implements a single-sample, one-hidden-unit network with tanh activation. Sample $x=1$, target $y=2$; $h=\tanh(wx+b)$, $\hat{y}=vh+c$ and $J=\frac{1}{2}(\hat{y}-2)^2$. Initial parameters are $w=0.5$, $b=0$, $v=1$, $c=0$ and learning rate $\eta=0.1$.

7.1 Forward and Backward Pass

Initially $h=0.4621171573$, $\hat{y}=0.4621171573$ and $J=1.1825418190$. Gradients: $\partial J/\partial w=-1.2094644752$, $\partial J/\partial b=-1.2094644752$, $\partial J/\partial v=-0.7106820475$ and $\partial J/\partial c=-1.5378828427$. All updates use gradients evaluated at the initial parameter values.

Parameter	Initial	Gradient	Updated
w	0.5	−1.2094644752	0.6209464475
b	0	−1.2094644752	0.1209464475
v	1	−0.7106820475	1.0710682047
c	0	−1.5378828427	0.1537882843

The required two-page handwritten derivation is supplied separately. The application also plots loss during iterative training.

7.2 Numerical Gradient Verification

Analytical gradients are checked against central finite differences with $\varepsilon=10^{-6}$. Reported absolute differences are approximately 10^{-10}, within the 10^{-5} tolerance. This validates the tested sample and parameter values, not every possible input.

8. Software Architecture and Reliability

A local HTTP API separates frontend requests from computation. C++17 owns geometry parsing, thermal solving, benchmark execution, mock hardware state and backpropagation. Background jobs report progress and support cancellation.

8.1 Job Management

The job manager uses QUEUED, RUNNING, COMPLETED, FAILED and CANCELLED states. Thermal and performance jobs use asynchronous endpoints. The queue is bounded; excess work is rejected. Job history is memory-only and pruned after limited retention, so it does not survive backend termination.

8.2 Validation

- Thermal inputs are checked for finite values, positive dimensions and conductivity, supported mesh sizes and valid boundary temperatures.
- Limits: up to 1,000 elements per mesh and 16 meshes per study.
- Invalid asynchronous thermal requests are rejected before job creation.
- Cancellation is checked during supported long-running operations; negative heat generation remains supported.

9. Testing and Verification

The latest supplied regression summary reports a Windows Ninja Release build, nine passing CTest tests, frontend type-check/lint/build success and static export of five routes. It also reports geometry fixtures, thermal mesh cases, invalid-input rejection, asynchronous thermal and benchmark completion, mock hardware operations, backpropagation and finite-difference checks.

The recorded CTest log independently confirms nine passing tests with zero failures. Frontend and live API results are based on the candidate-supplied regression summary. Extracted-package checks reportedly confirmed launch, loopback health and representative module operations. A clean Windows/offline installation test was not completed.

9.1 Release Packaging

The release package includes launcher, backend executable, static web assets, sample data and required non-Windows runtime DLLs. The packaging script inspects executable imports and copies compiler runtime DLLs. A fresh extraction test outside the repository was reported successful.

The final submission archive must also include source, tests, build configuration, documentation and version-control evidence. A runtime-only ZIP is not sufficient for the complete assignment.

10. Results and Limitations

- Hardware adapters are mocks; physical MCU, FPGA and FTDI communication has not been tested.
- The temperature CSV is a synthetic fixture; do not label it a verified ANSYS export without evidence.
- The STL sample's original creator/source should be stated only if confirmed.
- Job history is in-memory only and is not crash-persistent.
- Performance is workload- and machine-dependent; small parallel workloads may be slower.
- Clean Windows 11/offline installation was not completed.
- Audit the final ZIP for all required source, tests, documentation and version-control evidence.

10.1 Future Work

Future improvements could include support for additional geometry and solver file formats. Job management could be enhanced to retain previous simulation results. Hardware simulation could be expanded to cover more device types. These additions would make ENGINEX more flexible for engineering applications. They could also improve the overall user experience.

Further performance optimisation could improve execution for larger workloads. Enhanced error handling would make troubleshooting easier. Broader automated testing could increase application reliability. Testing across different hardware configurations would help identify performance limitations. These improvements could guide future development of the platform.

11. Conclusion

ENGINEX Workbench is a modular engineering simulation platform built with a C++17 backend and a static web interface. Its main features include geometry processing, thermal analysis, performance benchmarking, hardware simulation and backpropagation. Testing includes analytical thermal comparisons, mesh refinement and numerical gradient verification.

The application also demonstrates serial and parallel execution with numerical consistency checks. Automated tests validate important backend functionality and application workflows. Hardware integration remains simulated, while packaging and sample-data provenance have documented limitations.